DATE: 15-10-2026

CO-1 ASSESSMENT TOOL-3

(CODING CHALLENGE QUESTIONS)

COURSE CODE:DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

NAME: P.GAYATHRI REG.NO:192425277

Q-1

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

ANSWER;

CODE:

```
import re

email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])[A-Za-z\d@#$%&!]{8,}$"
mobile_pattern = r"^[6-9]\d{9}$"

email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
```

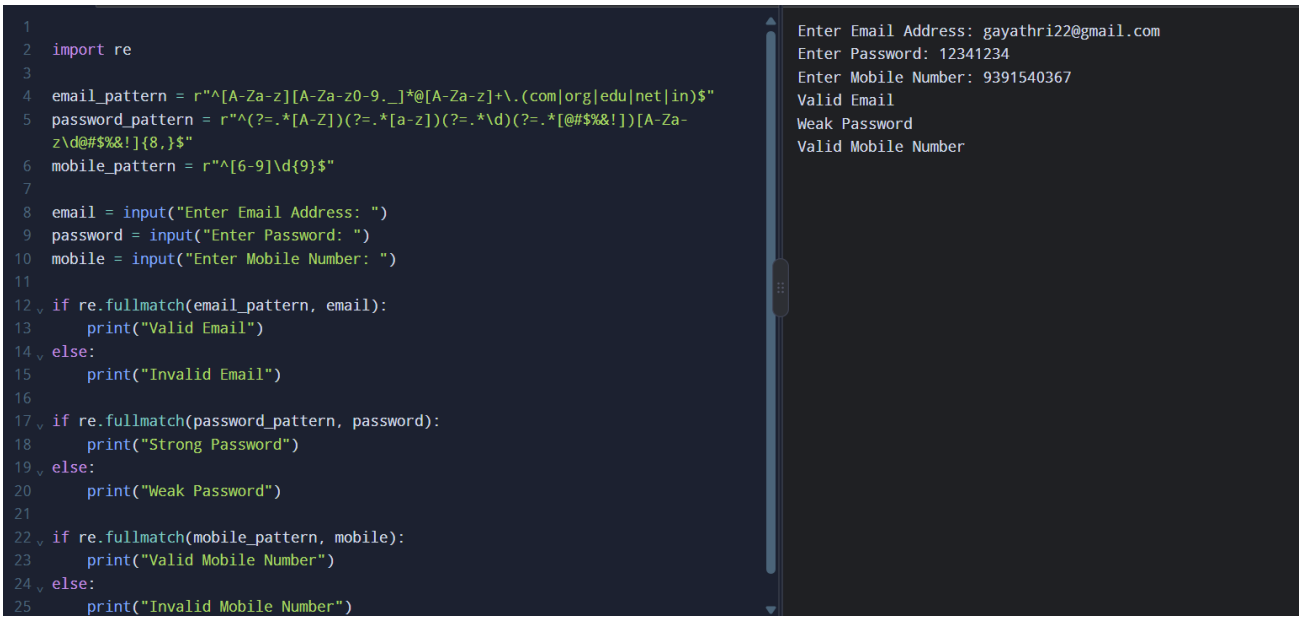
```
else:
    print("Invalid Mobile Number")
```

INPUT:

Enter Email Address: gayathri22@gmail.com
Enter Password: 12341234
Enter Mobile Number: 9391540367

OUTPUT:

Valid Email
Weak Password
Valid Mobile Number



```
import re

email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])[A-Za-z\d@#$%&!]{8,}$"
mobile_pattern = r"^[6-9]\d{9}$"

email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

```
Enter Email Address: gayathri22@gmail.com
Enter Password: 12341234
Enter Mobile Number: 9391540367
Valid Email
Weak Password
Valid Mobile Number
```

Q-2:

Section 2: Design a Finite State Automata (FSA) Simulator
2. Problem Statement
Develop a Python-based Deterministic Finite Automata (DFA) simulator.

ANSWER:

CODE:

```
# DFA Simulator

states = input("Enter states (space separated): ").split()
alphabet = input("Enter input alphabet (space separated): ").split()

transition = {}
```

```
print("Enter transition table:")
for state in states:
    transition[state] = {}
    for symbol in alphabet:
        next_state = input(f"Transition from {state} on {symbol}: ")
        transition[state][symbol] = next_state

initial_state = input("Enter initial state: ")
final_states = input("Enter final states (space separated): ").split()

n = int(input("Enter number of input strings: "))

for _ in range(n):
    string = input("Enter input string: ")
    current_state = initial_state
    path = [current_state]
    valid = True

    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break

        current_state = transition[current_state][symbol]
        path.append(current_state)

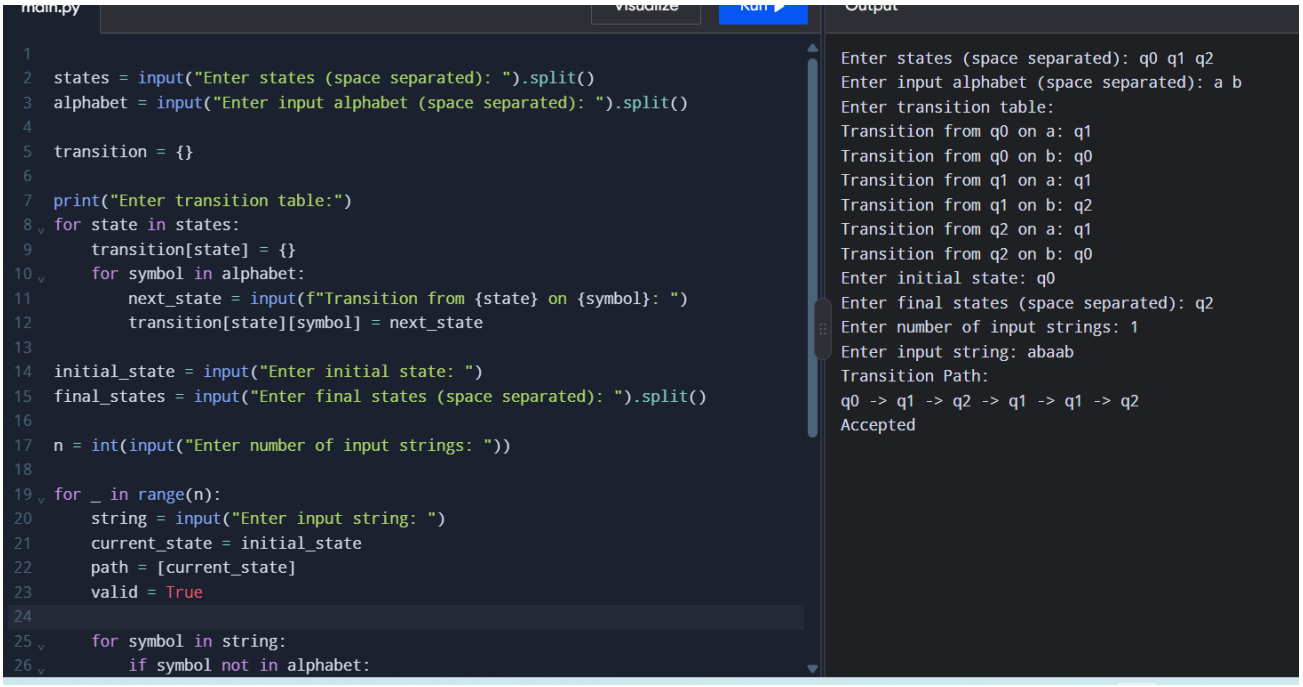
    print("Transition Path:")
    print(" -> ".join(path))

    if valid and current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

INPUT:

```
Enter states (space separated): q0 q1 q2
Enter input alphabet (space separated): a b
Enter transition table:
Transition from q0 on a: q1
Transition from q0 on b: q0
Transition from q1 on a: q1
Transition from q1 on b: q2
Transition from q2 on a: q1
Transition from q2 on b: q0
Enter initial state: q0
Enter final states (space separated): q2
Enter number of input strings: 1
Enter input string: abaab
```

OUTPUT:
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted



Q3:

Section 3: Smart Pattern Matching Engine using Regular Expressions
3. Problem Statement
Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

ANSWER:

CODE:

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Word")
    print("2. Search Date")
```

```
print("3. Search Phone Number")
print("4. Search Hashtag")
print("5. Search Mention")
print("6. Search Prefix")
print("7. Search Suffix")
print("8. Exit")

choice = input("Enter your choice: ")

if choice == "1":
    word = input("Enter word to search: ")
    pattern = r"\b" + re.escape(word) + r"\b"

elif choice == "2":
    pattern = r"\b\d{2}/\d{2}/\d{4}\b"

elif choice == "3":
    pattern = r"\b[6-9]\d{9}\b"

elif choice == "4":
    pattern = r"#[A-Za-z0-9_]+"

elif choice == "5":
    pattern = r"@[A-Za-z0-9_]+"

elif choice == "6":
    prefix = input("Enter prefix: ")
    pattern = r"\b" + re.escape(prefix) + r"\w*"

elif choice == "7":
    suffix = input("Enter suffix: ")
    pattern = r"\w*" + re.escape(suffix) + r"\b"

elif choice == "8":
    print("Exiting program...")
    break

else:
    print("Invalid choice!")
    continue

matches = re.findall(pattern, text, re.IGNORECASE)

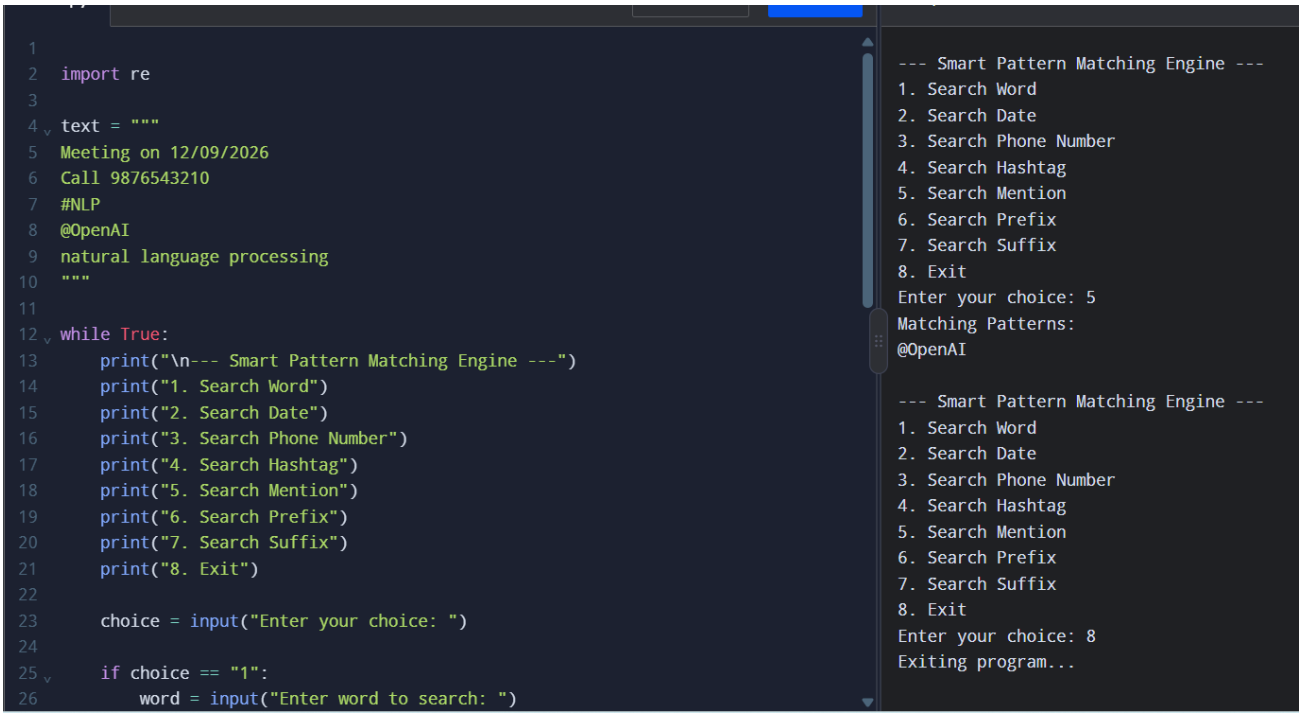
if matches:
    print("Matching Patterns:")
    for match in matches:
        print(match)
else:
    print("No match found.")
```

INPUT:

--- Smart Pattern Matching Engine ---
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
8. Exit
Enter your choice: 2

OUTPUT:

Matching Patterns:
12/09/2026



```python

import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Word")
    print("2. Search Date")
    print("3. Search Phone Number")
    print("4. Search Hashtag")
    print("5. Search Mention")
    print("6. Search Prefix")
    print("7. Search Suffix")
    print("8. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        word = input("Enter word to search: ")
```

```
--- Smart Pattern Matching Engine ---
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
8. Exit
Enter your choice: 5
Matching Patterns:
@OpenAI

--- Smart Pattern Matching Engine ---
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
8. Exit
Enter your choice: 8
Exiting program...
```